

```
int sum = 0;
// 计算从 1 到 10 (包含 10 在内) 的和
for (int val = 1; val <= 10; ++val)
    sum += val;    // 等价于 sum = sum + val
```

这种复合操作不仅对加法来说很常见,而且也常常应用于其他算术运算符或者 4.8 节(第 135 页)将要介绍的位运算符。每种运算符都有相应的复合赋值形式:

+=	-=	*=	/=	%=	// 算术运算符
<<=	>>=	&=	^=	=	// 位运算符, 参见 4.8 节 (第 135 页)

任意一种复合运算符都完全等价于

```
a = a op b;
```

唯一的区别是左侧运算对象的求值次数: 使用复合运算符只求值一次, 使用普通的运算符则求值两次。这两次包括: 一次是作为右边子表达式的一部分求值, 另一次是作为赋值运算的左侧运算对象求值。其实在很多地方, 这种区别除了对程序性能有些许影响外几乎可以忽略不计。

4.4 节练习

练习 4.13: 在下述语句中, 当赋值完成后 *i* 和 *d* 的值分别是多少?

```
int i; double d;
(a) d = i = 3.5;    (b) i = d = 3.5;
```

练习 4.14: 执行下述 *if* 语句后将发生什么情况?

```
if (42 = i) // ...
if (i = 42) // ...
```

练习 4.15: 下面的赋值是非法的, 为什么? 应该如何修改?

```
double dval; int ival; int *pi;
dval = ival = pi = 0;
```

练习 4.16: 尽管下面的语句合法, 但它们实际执行的行为可能和预期并不一样, 为什么? 应该如何修改?

```
(a) if (p = getPtr() != 0)    (b) if (i = 1024)
```

4.5 递增和递减运算符

递增运算符(++)和递减运算符(--)为对象的加 1 和减 1 操作提供了一种简洁的书写形式。这两个运算符还可应用于迭代器, 因为很多迭代器本身不支持算术运算, 所以此时递增和递减运算符除了书写简洁外还是必须的。

递增和递减运算符有两种形式: 前置版本和后置版本。到目前为止, 本书使用的都是前置版本, 这种形式的运算符首先将运算对象加 1 (或减 1), 然后将改变后的对象作为求值结果。后置版本也会将运算对象加 1 (或减 1), 但是求值结果是运算对象改变之前那个值的副本:

```
int i = 0, j;
j = ++i;    // j = 1, i = 1: 前置版本得到递增之后的值
j = i++;    // j = 1, i = 2: 后置版本得到递增之前的值
```

这两种运算符必须作用于左值运算对象。前置版本将对象本身作为左值返回，后置版本则将对象原始值的副本作为右值返回。

建议：除非必须，否则不用递增递减运算符的后置版本

有 C 语言背景的读者可能对优先使用前置版本递增运算符有所疑问，其实原因非常简单：前置版本的递增运算符避免了不必要的工作，它把值加 1 后直接返回改变了的运算对象。与之相比，后置版本需要将原始值存储下来以便于返回这个未修改的内容。如果我们不需要修改前的值，那么后置版本的操作就是一种浪费。

对于整数和指针类型来说，编译器可能对这种额外的工作进行一定的优化；但是对于相对复杂的迭代器类型，这种额外的工作就消耗巨大了。建议养成使用前置版本的习惯，这样不仅不需要担心性能的问题，而且更重要的是写出的代码会更符合编程的初衷。



在一条语句中混用解引用和递增运算符

如果我们想在一条复合表达式中既将变量加 1 或减 1 又能使用它原来的值，这时就可以使用递增和递减运算符的后置版本。

举个例子，可以使用后置的递增运算符来控制循环输出一个 vector 对象内容直至遇到（但不包括）第一个负值为止：

```
auto pbeg = v.begin();  
// 输出元素直至遇到第一个负值为止  
while (pbeg != v.end() && *pbeg >= 0)  
    cout << *pbeg++ << endl; // 输出当前值并将 pbeg 向前移动一个元素
```

对于刚接触 C++ 和 C 的程序员来说，*pbeg++ 不太容易理解。其实这种写法非常普遍，所以程序员一定要理解其含义。

后置递增运算符的优先级高于解引用运算符，因此 *pbeg++ 等价于 *(pbeg++)。pbeg++ 把 pbeg 的值加 1，然后返回 pbeg 的初始值的副本作为其求值结果，此时解引用运算符的运算对象是 pbeg 未增加之前的值。最终，这条语句输出 pbeg 开始时指向的那个元素，并将指针向前移动一个位置。

149

这种用法完全是基于一个事实，即后置递增运算符返回初始的未加 1 的值。如果返回的是加 1 之后的值，解引用该值将产生错误的结果。不但无法输出第一个元素，而且更糟糕的是如果序列中没有负值，程序将可能试图解引用一个根本不存在的元素。

建议：简洁可以成为一种美德

形如 *pbeg++ 的表达式一开始可能不太容易理解，但其实这是一种被广泛使用的、有效的写法。当对这种形式熟悉之后，书写

```
cout << *iter++ << endl;
```

要比书写下面的等价语句更简洁、也更少出错

```
cout << *iter << endl;  
++iter;
```

不断研究这样的例子直到对它们的含义一目了然。大多数 C++ 程序追求简洁、摒弃冗长，因此 C++ 程序员应该习惯于这种写法。而且，一旦熟练掌握了这种写法后，程序出错的可能性也会降低。

运算对象可按任意顺序求值

大多数运算符都没有规定运算对象的求值顺序（参见 4.1.3 节，第 123 页），这在一般情况下不会有什么影响。然而，如果一条子表达式改变了某个运算对象的值，另一条子表达式又要使用该值的话，运算对象的求值顺序就很关键了。因为递增运算符和递减运算符会改变运算对象的值，所以要提防在复合表达式中错用这两个运算符。

为了说明这一问题，我们将重写 3.4.1 节（第 97 页）的程序，该程序使用 for 循环将输入的第一个单词改成大写形式：

```
for (auto it = s.begin(); it != s.end() && !isspace(*it); ++it)
    *it = toupper(*it);    // 将当前字符改成大写形式
```

在上述程序中，我们把解引用 `it` 和递增 `it` 两项任务分开来完成。如果用一个看似等价的 `while` 循环进行代替

```
// 该循环的行为是未定义的！
while (beg != s.end() && !isspace(*beg))
    *beg = toupper(*beg++); // 错误：该赋值语句未定义
```

将产生未定义的行为。问题在于：赋值运算符左右两端的运算对象都用到了 `beg`，并且右侧的运算对象还改变了 `beg` 的值，所以该赋值语句是未定义的。编译器可能按照下面的任意一种思路处理该表达式：

```
*beg = toupper(*beg);    // 如果先求左侧的值
*(beg + 1) = toupper(*beg); // 如果先求右侧的值
```

也可能采取别的什么方式处理它。

4.5 节练习

150

练习 4.17：说明前置递增运算符和后置递增运算符的区别。

练习 4.18：如果第 132 页那个输出 `vector` 对象元素的 `while` 循环使用前置递增运算符，将得到什么结果？

练习 4.19：假设 `ptr` 的类型是指向 `int` 的指针、`vec` 的类型是 `vector<int>`、`ival` 的类型是 `int`，说明下面的表达式是何含义？如果有表达式不正确，为什么？应该如何修改？

```
(a) ptr != 0 && *ptr++          (b) ival++ && ival
(c) vec[ival++] <= vec[ival]
```

4.6 成员访问运算符

点运算符（参见 1.5.2 节，第 21 页）和箭头运算符（参见 3.4.1 节，第 98 页）都可用于访问成员，其中，点运算符获取类对象的一个成员；箭头运算符与点运算符有关，表达式 `ptr->mem` 等价于 `(*ptr).mem`：

```
string s1 = "a string", *p = &s1;
auto n = s1.size();    // 运行 string 对象 s1 的 size 成员
n = (*p).size();       // 运行 p 所指对象的 size 成员
n = p->size();          // 等价于 (*p).size()
```